

# Project 3: Perception-Guided Arm Control in Unstructured Scenes

Dimitrios Giannopoulos  
Georgios Paspalakis

June 29, 2026

## Abstract

This report presents the design, implementation, and evaluation of a perception-guided manipulation system for the Elephant Robotics MyArm 300 Pi 7-DOF robotic arm. The task is to detect task-relevant objects in an unstructured workspace, estimate their 3D position relative to the robot, generate arm motions, and adapt to perception and calibration errors. The implemented system uses an Intel RealSense RGB-D camera for visual sensing, ArUco-based camera-to-world calibration, HSV and depth-based segmentation for target cube detection, top-face estimation from 3D point clouds, optimization-based inverse kinematics, and TCP-based robot command transmission. The system was developed around a suction cup end-effector, with the main manipulation objective being to reliably identify, grasp and relocate the target cubes.

# Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>System Overview</b>                                   | <b>3</b>  |
| <b>2</b> | <b>Hardware Setup</b>                                    | <b>3</b>  |
| 2.1      | Robot Arm . . . . .                                      | 3         |
| 2.2      | RGB-D Camera . . . . .                                   | 3         |
| 2.3      | Vacuum Sucker Kit and Relay Module . . . . .             | 4         |
| 2.4      | 3D Printed Base . . . . .                                | 5         |
| <b>3</b> | <b>Perception Pipeline</b>                               | <b>6</b>  |
| 3.1      | Coordinate Frames and Calibration . . . . .              | 6         |
| 3.2      | RGB-D Cube Detection and Top-Face Estimation . . . . .   | 7         |
| 3.3      | Visual Servoing Perception . . . . .                     | 9         |
| <b>4</b> | <b>Motion Planning and Control Pipeline</b>              | <b>10</b> |
| 4.1      | Kinematic Model . . . . .                                | 10        |
| 4.2      | Inverse Kinematics . . . . .                             | 10        |
| 4.2.1    | QP-Based Inverse Kinematics . . . . .                    | 10        |
| 4.2.2    | Newton–Raphson Jacobian IK for Visual Servoing . . . . . | 12        |
| 4.3      | Task Automation Pipeline . . . . .                       | 12        |
| <b>5</b> | <b>System Integration</b>                                | <b>13</b> |
| 5.1      | TCP Communication . . . . .                              | 13        |
| 5.2      | ROS 2 Node Graph . . . . .                               | 14        |
| <b>6</b> | <b>Results and Discussion</b>                            | <b>15</b> |

# 1 System Overview

The goal of this project is to build a robotic manipulation system capable of autonomously detecting, localizing, and picking task-relevant objects (red cubes) from an unstructured workspace that also contains non-target objects. The system observes the scene through a mounted RGB-D camera, identifies only the target objects, estimates their 3D position relative to the robot base frame, generates arm motions accordingly, and adapts online to residual perception and calibration errors. The end-effector is a suction cup, and the manipulation objective is to position the suction tip precisely above the top face of a target cube, descend to make contact, and relocate the cube to a deposit location.

Task-relevant objects are detected and localized from RGB-D sensing by isolating the red cubes from the rest of the cluttered scene through HSV color segmentation, while non-target objects are ignored (Section 3.2). Their pose is estimated relative to the robot by combining an ArUco-based extrinsic calibration, which places every camera-frame measurement into the robot base (world) frame, with a table-anchored top-face estimator that returns each cube’s center in world coordinates (Sections 3.1–3.2). Arm motions are then generated from the perceived scene by a multi-start QP inverse-kinematics solver, which computes a limit-aware pre-grasp pose with the tool axis pointing downward and executes it on the 7-DOF arm (Section 4.2.1). The system adapts online to perception and calibration errors through a one-shot visual-servoing step that measures the residual XY misalignment between the cube and the suction cup directly from the live image and corrects it before contact, backed by a cube-count grasp check that triggers retries on failure (Sections 3.3 and 4.3). Finally, obstacles are avoided and robot constraints respected by enforcing joint-position and step-size limits inside both IK solvers, aborting any move whose joint change exceeds a safe threshold, and lifting the end-effector clear of neighbouring cubes before transport (Sections 4.2.1–4.2.2 and 4.3).

The system is implemented as a ROS 2 workspace with two packages: `perception_pkg` (camera calibration, cube detection, visual servoing) and `control_pkg` (IK solvers, TCP bridge, grasp sequencer). All the computationally heavy vision processing is performed on the laptop. The Pi runs two lightweight TCP servers and is responsible only for the low-level hardware interface: it receives and executes joint angle commands, reports the arm’s current joint angles when queried, and sets the GPIO pins that drive the suction kit’s pump and valve relays.

## 2 Hardware Setup

### 2.1 Robot Arm

The robotic arm used in this task is the Elephant Robotics MyArm 300 Pi. It is a compact 7-DOF serial robotic arm with a maximum reach of 300 mm and a payload capacity of 200 g.

The arm’s main controller is a Raspberry Pi 4B running Elephant Robotics’ customized Ubuntu Mate 20.04, which is embedded in the base. This makes the arm self-contained: it requires no external PC to operate and comes pre-loaded with ROS 2, Python, and the `pymycobot` library for control. The Raspberry Pi communicates with the arm’s joint servos over serial and exposes network interfaces that allow the arm to be controlled remotely from a laptop.

### 2.2 RGB-D Camera

The scene is observed using an Intel RealSense D435i RGB-D camera. The D435i provides two synchronized streams: a color stream at up to  $1920 \times 1080$  resolution and a depth stream at up to  $1280 \times 720$  resolution. In this project both are run at their full resolution ( $1920 \times 1080$  color,  $1280 \times 720$  depth) at 30 FPS. The depth stream is produced by an active infrared stereo pair and is hardware-aligned to the color frame so that each color pixel has a corresponding depth value in meters.

The camera also includes an onboard IMU, which is not used in this project. Factory-calibrated intrinsic parameters (focal lengths  $f_x$ ,  $f_y$  and principal point  $c_x$ ,  $c_y$ ) are stored on the device and accessed at runtime. All camera interaction — stream configuration, frame acquisition, depth filtering, intrinsic retrieval, and depth-to-color alignment — is performed through the `pyrealsense2` Python library.

### 2.3 Vacuum Sucker Kit and Relay Module

The vacuum sucker kit is a common pump–valve–suction cup system used for lightweight pick-and-place applications. When the pump is activated and the valve remains closed, negative pressure is generated inside the tube and suction cup, allowing the system to grip the object. To release the object, the electronic valve is activated, which vents the tube and removes the vacuum, causing the object to detach from the suction cup. This specific kit is intended for small loads and can transport objects up to approximately 60 g, depending on the surface quality and how well the suction cup seals against the object.

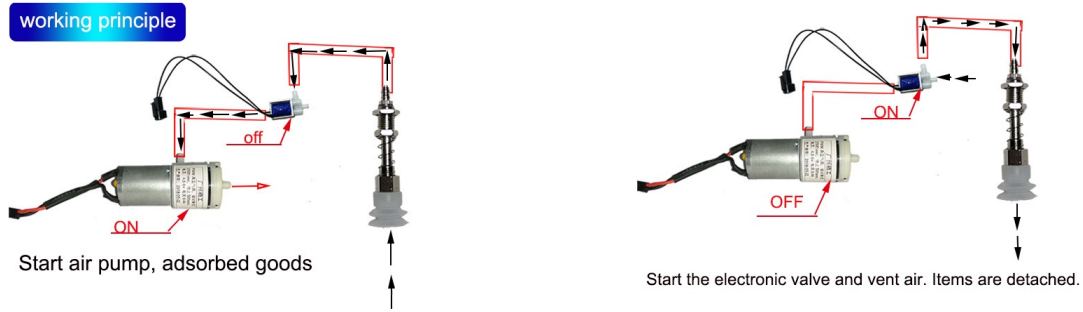


Figure 1: Vacuum sucker kit working principle.

Interfacing the vacuum sucker kit with the robot’s Raspberry Pi was not straightforward. According to its datasheet, the pump is rated at 5 V with an idle current of 350 mA, while the valve is rated at 5 V drawing 220 mA. Powering both components from a common 5 V supply therefore implies a combined idle draw of approximately 570 mA. Driving these components directly from the Raspberry Pi was not feasible for two reasons:

1. The Raspberry Pi GPIO pins operate at 3.3 V logic, whereas the kit’s components require 5 V.
2. Each of the 40 GPIO pins can output at most 17 mA, with a maximum combined current across all pins of 51 mA. This is more than ten times less than the current required by the kit.

Several solutions were considered, including a logic-level MOSFET switching circuit and dedicated motor drivers. We ultimately selected a relay module, specifically an active-low, 5 V, two-channel relay module with a separate relay-coil supply line.

A relay module allows a low-power control signal (e.g. a pin output from a microcontroller) to switch the connection between two terminals on the isolated output side. The selected module is rated for terminal loads of up to 10 A at 30 V DC, so the 0.35 A and 0.22 A loads at 5 V DC are well within its capability.

A two-channel relay module contains two independent relays, each driven by its own signal input. Our module is active-low: when the input pin is pulled LOW, current flows through the input-side LED of the optocoupler. This activates the optocoupler’s phototransistor, which drives the coil-side transistor and allows current to flow through the relay coil. The energized coil generates a magnetic field that displaces a contact armature, switching the output connection. In the de-energized state, COM is connected to NC (Normally Closed); when the coil is energized, COM disconnects from NC and connects to NO (Normally Open).

Each relay coil draws approximately 67 mA at 5 V when energized, so with both the pump and valve relays active the coils alone draw about 134 mA. In a typical single-supply relay module, a single VCC line powers both the relay coil and the input/trigger circuit that detects the IN signal level. If such a module were powered entirely at 5 V, its input circuit would be referenced to 5 V, and a 3.3 V Raspberry Pi GPIO output might neither reliably trigger the relay nor be safely interfaced.

For this reason, we selected a relay module with separate supply lines for the relay coils and the signal/control side. These modules usually come with a jumper that connects the relay-coil supply line to the signal supply line by default. By removing this jumper, we can supply the relay coils with 5 V while powering the signal/control side with 3.3 V. This allows the relays to operate at their proper coil voltage while keeping the input compatible with the Raspberry Pi's 3.3 V GPIO logic.

In our application, the relay module is used to break the connection between the supply positive rail and (i) the pump positive terminal and (ii) one of the two valve leads (the valve is non-polarized). The connections are as follows: (Explain wiring in a photo or in a table)

**Note:** With the configuration described above, the measured current drawn from the power supply is approximately 50 mA per energized relay, instead of the rated 67 mA at 5 V. According to the datasheet for these relays, the minimum coil current required for reliable actuation is approximately 55 mA. During extensive testing, we have not observed any failed switching events. The relay contacts have always changed state correctly when commanded. However, the measured current is slightly below the datasheet value for reliable actuation, which suggests that this configuration may have limited operating margin. Therefore, although it works in practice, it may not be the most robust long-term solution for interfacing the vacuum sucker kit with the Raspberry Pi.

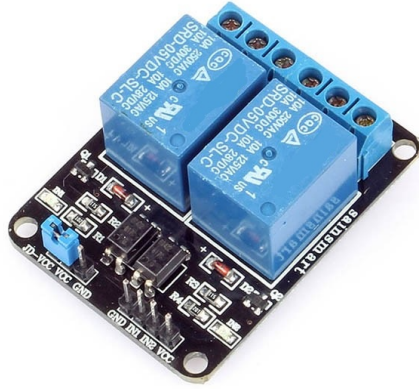


Figure 2: Two-channel relay module used to control the pump and valve.

## 2.4 3D Printed Base

To mount the suction kit onto the robotic arm, a custom base was designed and 3D printed. The base was modeled in Autodesk Fusion and split into two halves, each printed separately. This two-part construction allows the pneumatic tube to be routed through the interior of the base, ensuring that the suction cup remains fully centered with respect to the base frame, with the only displacement being an offset along the z-axis. The two halves are joined using two screws and nuts seated in dedicated holes, and the assembled base is fastened to the robot through the four mounting holes provided on the end-effector flange.

The current design leaves room for improvement, primarily by minimization the overall base length. The z-axis offset of the suction cup’s edge center point, measured at 14.6 cm for the current design, is incorporated into the inverse kinematics as the tool length offset. The larger this offset, the smaller the resulting dexterous workspace of the arm. Therefore, reducing the base length directly enlarges the space around the robot where the end-effector can be arbitrarily oriented.

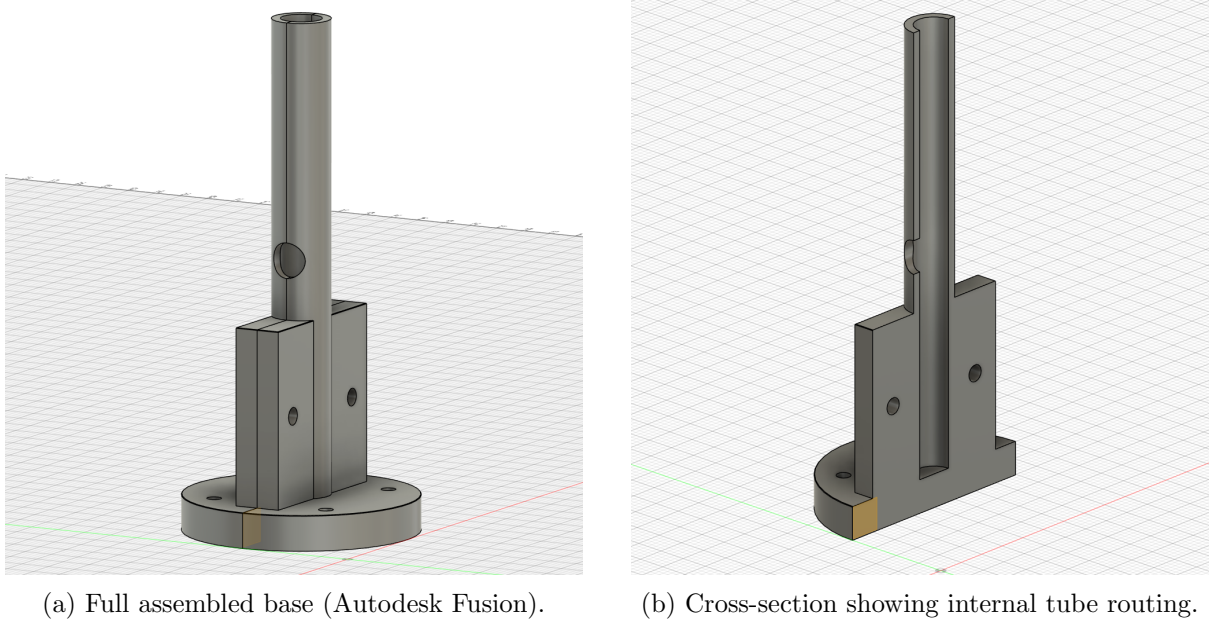


Figure 3: 3D printed suction cup base design.

### 3 Perception Pipeline

#### 3.1 Coordinate Frames and Calibration

The system operates across three coordinate frames:

- **Camera frame  $C$ :** the RealSense optical frame, origin at the camera lens. Depth values are expressed in this frame.
- **World frame  $W$ :** the robot base frame, used for all IK targets.

Perception produces 3D object positions in the camera frame. These must be transformed into the world frame before IK can be solved. This transform is obtained through extrinsic calibration.

##### Camera Intrinsics

The RealSense SDK provides factory-calibrated intrinsics. For a pixel  $(u, v)$  with depth  $z$ , the corresponding camera-frame 3D point is:

$$x_C = \frac{(u - c_x)z}{f_x}, \quad y_C = \frac{(v - c_y)z}{f_y}, \quad z_C = z.$$

##### ArUco Extrinsic Calibration

An ArUco marker is placed at a known world position. The calibration node `aruco_extrinsic_calibrator.py` detects the marker in successive frames, estimates its 6-DOF camera-frame pose using `cv2.solvePnP`,

and accumulates samples over a fixed window. The averaged result gives the camera-to-world homogeneous transform:

$$T_{WC} = \begin{bmatrix} R_{WC} & t_{WC} \\ \mathbf{0} & 1 \end{bmatrix}.$$

Any camera-frame point  $p_C$  is then transformed into world frame by:

$$p_W = R_{WC} p_C + t_{WC}.$$

The result is saved to `~/ros/myarm_camera_extrinsic.json` and loaded by all subsequent perception nodes. Calibration is re-run whenever the camera is physically moved.

In practice this process is necessary if we want millimeter-level accuracy for the extrinsic calibration, which is essential for this manipulation task: an error of even 5 mm could mean that the robot fails to grasp the cube.

### 3.2 RGB-D Cube Detection and Top-Face Estimation

The core perception task is to identify the top face of a red cube and return its center in world coordinates. The method is implemented in `top_face_center_mask.py` and is deliberately robust to depth noise by anchoring the top-face height to the table plane rather than relying on noisy per-pixel depth values.

#### RGB-D Acquisition

Frames are acquired directly via `pyrealsense2`. The color and depth streams are hardware-aligned so each color pixel has a valid depth reading. The configured streams are:

- Color:  $1920 \times 1080$  at 30 FPS, BGR.
- Depth:  $1280 \times 720$  at 30 FPS, aligned to color.

Although the D435i depth stream supports up to 90 FPS, 30 FPS was sufficient for this application since perception is performed on a static scene with the arm at rest.

Spatial, temporal, and hole-filling filters are applied to the depth stream to reduce noise before further processing.

#### Red Cube Segmentation

The cube is segmented in HSV color space. Because red wraps around the hue boundary in OpenCV ( $H \in [0, 179]$ ), two ranges are combined:

$$H \in [0, 10] \cup [170, 179].$$

Morphological opening and closing are applied to remove isolated noise pixels and fill small holes in the mask. A minimum connected-component area threshold discards spurious detections.

An important practical limitation we encountered is that the HSV thresholds for the red cubes are not a one-time calibration. The segmentation is highly sensitive to the lighting conditions in the workspace, because the measured hue, saturation, and value of the cube surface depend directly on the illumination. We found that the thresholds systematically drift out of tune whenever the lighting changed: moving to a different workbench, switching the room lights on or off, or even the gradual change in ambient daylight as it got darker outside were all enough to degrade the mask, causing the cube to be partially missed or the background to be picked up as false detections. In practice the thresholds therefore have to be re-tuned for each new setup and re-checked whenever the lighting changes. To make this practical, the detector

exposes the saturation and value thresholds as live trackbars (see Section 3.2), so the mask can be re-tuned in seconds against the live camera feed rather than by editing and restarting the code. A more robust long-term solution would be to reduce this dependence on absolute color values, for example by controlling the workspace illumination or by adopting a lighting-invariant segmentation approach.

### Table-Anchored Top-Face Estimation

Rather than using noisy depth values for each individual pixel, the algorithm estimates the table surface height from depth readings in a region around the cube, then derives the top-face height analytically. A cube does not necessarily rest directly on the table: cubes may be stacked, so the top face of interest can sit one or more cube heights above the table. The estimator therefore does not assume a single fixed height  $z_{\text{table}} + h_{\text{cube}}$ , but instead determines which stack level a cube belongs to.

For a stack of integer level  $k$  (with  $k = 1$  for a cube on the table,  $k = 2$  for a cube resting on top of one other cube, and so on), the top face lies on the horizontal plane

$$z_{\text{top}}(k) = z_{\text{table}} + k h_{\text{cube}},$$

where  $h_{\text{cube}} = 2.4\text{ cm}$  is the known cube edge length. To select the correct level, the algorithm measures how high the red blob actually reaches in world coordinates. It computes a high percentile (the 95<sup>th</sup>) of the world-frame heights of the cube’s red pixels,  $z_{\text{red,top}}$ , which corresponds to the visible top of the blob while rejecting outliers, and snaps it to the nearest level:

$$k = \text{round}\left(\frac{z_{\text{red,top}} - z_{\text{table}}}{h_{\text{cube}}}\right), \quad k \leftarrow \text{clip}(k, 1, k_{\text{max}}),$$

with  $k_{\text{max}} = 3$  stack levels supported. The estimated top-face height is then  $z_{\text{top}} = z_{\text{top}}(k)$ . This makes the detector report each cube’s true top face rather than incorrectly assuming every cube sits on the table.

It is worth noting that the dynamic table height estimation is implemented for generality and robustness across different setups. In our specific deployment, the robot is bolted to the table and the camera position is fixed, meaning the table world-frame height  $z_{\text{table}}$  is a constant. Hardcoding this value would make the detector robust even in heavily occluded scenes where the surrounding table surface is not visible, at the cost of losing generality across setups.

Red pixels whose world-frame  $z$  coordinate falls within a tolerance band around the selected  $z_{\text{top}}$  are classified as top-face pixels:

$$-\delta_{\text{below}} < z_W - z_{\text{top}} < \delta_{\text{above}}, \quad \delta_{\text{below}} = 6\text{ mm}, \quad \delta_{\text{above}} = 8\text{ mm}.$$

These top-face pixels are projected onto the horizontal plane at  $z_{\text{top}}$  by ray-casting from the camera center through each pixel, intersecting with the known plane. This eliminates depth noise from the top-face position estimate.

### Top-Face Center Estimation

The 2D footprint of the top-face pixels on the  $z_{\text{top}}$  plane is used to fit a  $2.4\text{ cm} \times 2.4\text{ cm}$  square. The center of this square, expressed in world coordinates  $(x_W, y_W, z_{\text{top}})$ , is the suction target point. The result is averaged over 2 seconds of frames and published as a `PointStamped` message on `/perception/suction_point_world` once the standard deviation across frames drops below 2 cm.

### 3.3 Visual Servoing Perception

After the arm reaches the pre-grasp pose (approximately 5 cm above the cube), an XY misalignment remains. We believe that this misalignment is mainly due to the mechanical looseness of the robot and the non-perfect top-face center estimation algorithm. A visual servoing step is therefore necessary to measure this error directly and command the robot to make a corrective motion.

#### Dual-Target Detection

The node `visual_servoing_perception_node.py` detects two targets simultaneously in each frame:

- **Cube top-face center:** detected using the table-anchored method explained in Section 3.2.
- **End-effector mask center:** a piece of blue tape attached to the suction cup is segmented using HSV. The range used is approximately  $H \in [94, 105]$ ,  $S \in [55, 140]$ ,  $V \in [95, 155]$ , but, as with the red cube, this may also need recalibration if the lighting conditions change. Afterwards, a minimum-area filter discards small noise blobs, and the algorithm keeps only the single remaining blob whose centroid is nearest to the detected cube top-face center. The mask’s center is deprojected to world coordinates using the aligned depth frame and  $T_{WC}$ .

#### Error Computation and Command

Having acquired these two world points, the XY correction can be computed. The objective of visual servoing is to make the two centers share the same  $x$  and  $y$  coordinates, up to a small fixed offset: the 3D point the depth camera reads on the blue tape lies on the edge of the cylindrical mounting mechanism rather than on its axis, so to recover the true dead-center of the suction cup the measured end-effector point is offset along world  $X$  by approximately the radius of the mount (0.4 cm). Writing  $\bar{p}_{\text{cube},W}$  for the averaged cube top-face center and  $\bar{p}_{\text{ee},W}$  for the averaged, offset-corrected end-effector center, the correction is

$$\Delta x = \bar{x}_{\text{cube},W} - \bar{x}_{\text{ee},W}, \quad \Delta y = \bar{y}_{\text{cube},W} - \bar{y}_{\text{ee},W}.$$

Both centers are averaged over a 3-second window with a 2-second warmup to allow the blue end-effector mask to settle. Our experiments showed that for roughly the first two seconds the algorithm consistently mis-masked the end-effector and picked up background noise, so these early frames are discarded. The averaged correction is published as a `Vector3` on `/visual_servoing/move_cm` and consumed by the control pipeline, together with a fixed  $z$  descent that brings the suction cup down into contact with the cube. This is a one-shot correction, not a continuous servo loop.

#### Hardcoded Corrective Offsets

After performing dozens of visual servoing tests, we found that the resulting end-effector position was still not centered well enough for reliable grasping: it was systematically landing on the top-right corner of the cube rather than its center. Rather than chase this through further re-calibration, we measured this repeatable, systematic error and hardcoded it as a set of extra offsets added on top of the computed  $\Delta x$  and  $\Delta y$  before they enter the inverse-kinematics solver for the servoing move. In our final configuration these extras are  $\Delta x_{\text{extra}} = 0$ ,  $\Delta y_{\text{extra}} = -1$  cm, and an additional  $\Delta z_{\text{extra}} = -2$  cm of descent on top of the nominal  $-3$  cm (so the cup descends 5 cm in total to ensure contact). This simple correction yielded a large improvement in the success rate of landing on the cube top-face center.

## 4 Motion Planning and Control Pipeline

### 4.1 Kinematic Model

The MyArm 300 Pi is modeled using the Product of Exponentials (POE) formulation. Each joint  $i$  is associated with a screw axis  $S_i$  expressed in the world frame at the robot home configuration. The end-effector pose for joint vector  $q \in \mathbb{R}^7$  is:

$$T(q) = e^{[S_1]q_1} e^{[S_2]q_2} \dots e^{[S_7]q_7} M,$$

where  $M$  is the home-configuration end-effector pose. The home pose includes the physical suction tool length (14.6 cm) so that  $T(q)$  gives the position of the suction tip:

$$p_M = [0 \quad 0 \quad 47.88 + 14.6]^T \text{ cm}.$$

The spatial Jacobian  $J(q) \in \mathbb{R}^{6 \times 7}$  is computed analytically from the screw axes and used by both IK solvers. Joint limits are:

$$\begin{aligned} J_1: [-160, 160] & \quad J_5: [-170, 170] \\ J_2: [-70, 115] & \quad J_6: [-115, 115] \\ J_3: [-170, 170] & \quad J_7: [-180, 180] \\ J_4: [-113, 75] & \end{aligned}$$

### 4.2 Inverse Kinematics

The system uses two inverse-kinematics solvers, each suited to a different stage of the pick. A multi-start quadratic-programming (QP) solver computes the pre-grasp pose from scratch, where a globally good, limit-aware configuration matters more than speed. A lightweight damped-Jacobian (Newton–Raphson) solver then handles the small visual-servoing corrective motions, where the robot is already near the target and only a fast local refinement starting from the current configuration is needed. Both solvers share the kinematic model of Section 4.1.

#### 4.2.1 QP-Based Inverse Kinematics

Pre-grasp joint angles are computed by `qp_ik_node.py`, which calls the multi-start quadratic-programming IK solver in `qp_ik_solver.py`. The desired suction point is the detected cube top-face center raised by the pre-grasp clearance:

$$p_{\text{des}} = p_{\text{suction}, W} + \begin{bmatrix} 0 \\ 0 \\ 5 \text{ cm} \end{bmatrix}.$$

The solver must place the suction tip at  $p_{\text{des}}$  while keeping the tool  $z$ -axis pointing straight down ( $z_{\text{des}} = [0, 0, -1]^T$ ), respecting joint position limits and joint velocity limits. Rotation about the suction axis (yaw) is left free.

**Task terms.** Two tasks are stacked. The position task drives the suction point toward  $p_{\text{des}}$ : with position error  $e_p = p_{\text{des}} - p$  and desired Cartesian velocity  $t_p = e_p / \Delta t$ , the translational Jacobian rows  $J_p$  should satisfy  $J_p \dot{q} \approx t_p$ . The orientation task keeps the tool  $z$ -axis pointing down: from the current tool axis  $z_{\text{ee}}$  and the angular Jacobian rows  $J_\omega$ , the axis moves as  $\dot{z} = J_z \dot{q}$  with  $J_z = -[z_{\text{ee}}]_\times J_\omega$ , and the target  $t_z$  (from `z_axis_task`) rotates  $z_{\text{ee}}$  toward  $z_{\text{des}}$ . The two tasks, plus a small damping (regularization) term that keeps  $\dot{q}$  small and well-conditioned, are written as a single weighted least-squares objective  $\frac{1}{2} \|Wx - t\|^2$  with

$$W = \begin{bmatrix} w_p J_p \\ w_z J_z \\ \lambda I \end{bmatrix}, \quad t = \begin{bmatrix} w_p t_p \\ w_z t_z \\ 0 \end{bmatrix}.$$

**QP Formulation.** Expanding the least-squares objective,  $\frac{1}{2}\|Wx - t\|^2 = \frac{1}{2}x^T(W^TW)x - (W^Tt)^Tx + \text{const}$ , gives the standard form

$$\min_x \frac{1}{2} x^T Q x + q_{\text{lin}}^T x \quad \text{s.t.} \quad Cx - d \leq 0, \quad Q = W^TW, \quad q_{\text{lin}} = -W^Tt.$$

The inequality constraints stack the joint-limit and velocity-limit bounds. Joint limits  $q_{\min} \leq q + \Delta t \dot{q} \leq q_{\max}$  and velocity limits  $\dot{q}_{\min} \leq \dot{q} \leq \dot{q}_{\max}$  become

$$C = \begin{bmatrix} -\Delta t I \\ \Delta t I \\ -I \\ I \end{bmatrix}, \quad d = \begin{bmatrix} q - q_{\min} \\ q_{\max} - q \\ -\dot{q}_{\min} \\ \dot{q}_{\max} \end{bmatrix}.$$

There are no equality constraints. The QP is solved each iteration with the OSQP backend through `qpsoolvers`; iteration stops when the position error and the  $z$ -axis angle error both fall below their tolerances, which we currently set to 0.5 cm in position and  $0.5^\circ$  for the tool  $z$ -axis.

**Joint-limit margin.** A subtle but important requirement is that the pre-grasp configuration must not leave any joint sitting near the edge of its travel. The pre-grasp pose is only an intermediate stop: from it, the visual-servoing step (Section 3.3) commands a further corrective micro-move and descent. If a joint were already close to its limit at the pre-grasp pose, the small additional motion required for visual servoing could push it against the hard limit, and the corrective IK would fail to converge. To prevent this, the solver enforces a joint-limit margin: during the IK search the joint limits are artificially tightened by  $15^\circ$ , so the solver only accepts configurations that keep every joint at least  $15^\circ$  away from its true limit. In addition, among the configurations that do converge, the candidate-ranking score includes a joint-margin term (weight 0.5) that prefers solutions further from the limits. This leaves enough joint range for the visual-servoing correction to be carried out reliably afterward.

## Multi-start search

Because this project does not use trajectory planning, we do not need the IK to start from, or track, the robot’s current pose: we simply command a desired joint-angle target and let the robot’s own controller decide how to move there. What we care about is therefore not the initial conditions of the IK, but obtaining a fast, reliable QP IK convergence to a good final configuration. The solver exploits this freedom by searching for a good starting configuration rather than tracking one. It samples many random joint configurations within the (margined) joint limits, evaluates forward kinematics for each, and ranks them by how close they already are in position and tool-axis direction. It then runs the iterative QP IK from the best-ranked candidates and returns the first one that converges (or, if none do, the best-scoring failed attempt). This avoids the local minima a single fixed start would fall into. In the pre-grasp configuration, 96 candidates are sampled and the best 8 are used as independent starting points, with convergence tolerances of 0.5 cm in position and  $0.5^\circ$  for the tool  $z$ -axis.

The solved joint angles are published on `/myarm/joint.targets` and forwarded to the robot over TCP.

### 4.2.2 Newton–Raphson Jacobian IK for Visual Servoing

Small corrective motions (visual servoing micro-moves) are solved by `nr_ik.py` using a damped Jacobian iterative method. Starting from the robot’s current joint configuration  $q_0$  (read from the robot over TCP), the solver iterates:

$$\Delta q_k = J_{\text{task}}(q_k)^{\#} e_k,$$

where  $J_{\text{task}}^{\#}$  is the damped pseudo-inverse of the task Jacobian and  $e_k$  is the combined position and orientation error. The update rule uses damped least squares to avoid singularity:

$$J^{\#} = J^T(JJ^T + \lambda^2 I)^{-1}.$$

The method converges rapidly for small moves (a few centimetres) starting from a good initial configuration. Per-iteration joint step limits prevent large jumps. A safety check aborts the motion if any joint change exceeds 45 between the current and target configuration, preventing unsafe large motions caused by bad IK solutions.

This solver is used by both `visual_servoing_motion_node.py` (standalone visual servoing pipeline) and `task_automation_node.py` (automated multi-cube pipeline).

### 4.3 Task Automation Pipeline

For fully automated multi-cube picking, `task_automation_node.py` ties together perception, both IK solvers, and the suction hardware into a single orchestrating node. Rather than communicating through the ROS topic graph, it drives the RealSense camera and the two TCP servers (joint and suction) directly, so it is the only node that needs to run. This subsection describes the control flow; the perception and IK building blocks it calls are those of Sections 3 and 4.2.

**Cube detection and ordering.** Each round begins with the arm at its home configuration so that it does not occlude the table. The node detects every red cube top face (Section 3.2), averaging detections over several frames and clustering them so that only stable, repeatedly-seen cubes are kept. The detected cubes are then ordered: stacked cubes (higher world  $z$ ) are picked first so that removing a lower cube never destabilizes one above it, and within the same height level the cube with the most negative world  $y$  is taken first. The first cube in this order is the current target.

**Pre-grasp.** For the target cube, the desired suction point is raised 5 cm above the detected top-face center and the pre-grasp joint angles are computed with the multi-start QP solver (Section 4.2.1). If the QP fails, it is retried once; if it fails again, the cube is treated as out of reach and skipped. Otherwise the joint target is sent to the arm and the node waits for it to settle.

**Visual-servoing grasp.** Once at the pre-grasp pose, the suction pump is switched on so that contact immediately grips the cube. The node then gathers the visual-servoing XY correction (Section 3.3) over a short averaging window, and executes the resulting corrective micro-move together with the downward descent using the damped-Jacobian solver (Section 4.2.2). A per-move joint-delta safety check aborts the motion if any joint would change by more than a safe threshold, guarding against a bad IK solution.

**Lift, transport, and release.** After contact, the arm first lifts straight up on the world  $z$ -axis to clear any neighbouring cubes, then moves to a dedicated drop pose. The node confirms (by polling joint angles over TCP) that the drop pose is actually reached before releasing, so the cube is not dropped mid-transit. The cube is then released by switching the pump off and pulsing the valve open briefly to break the vacuum.

**Grasp verification and looping.** The grasp is verified by cube count rather than by any force or contact sensor: the node re-detects all cubes and checks whether the total has decreased by one. If it has, the grasp is confirmed and the loop moves on to the next cube. If not, the attempt is retried, up to a maximum of three attempts, after which the cube is skipped. Re-detecting the cubes at this point also produces the target list for the next round. The loop continues until no red cubes remain in the scene, at which point the arm returns home and the sequence ends.

## 5 System Integration

### 5.1 TCP Communication

Because the laptop and the MyArm Raspberry Pi ran different ROS 2 distributions during development, direct ROS 2 inter-machine communication was unreliable. A lightweight TCP bridge was implemented instead. The Pi runs two standalone Python TCP servers with no ROS dependency, and the laptop acts as the client for both.

All communication uses a simple line-delimited JSON protocol: the client sends one JSON object followed by a newline, and the server replies with one JSON object followed by a newline. Every response includes an `ok` field indicating success or failure.

#### Joint Server (port 5017)

The joint server runs `joint_tcp_server.py` on the Pi. It interfaces directly with the arm's servos over serial (`/dev/ttyAMA0`, 115200 baud) via the `pymycobot` library. On startup it moves the arm to the all-zeros home configuration. It handles three types of requests from the laptop:

- **Move to joint target.** The laptop sends a list of seven joint angles (either in degrees via `angles_deg` or in radians via `angles_rad`) together with a `speed` value (1–100). The server calls `pymycobot`'s `send_angles` over serial, which commands the arm's servos. A duplicate-detection mechanism silently accepts a command if it is within  $0.5^\circ$  of the last sent target, preventing unnecessary re-sends of the same pose.
- **Read current joint angles.** The laptop sends `"type": "get_angles"`. The server calls `pymycobot`'s `get_angles` to read the seven current joint positions from the arm and returns them in both degrees and radians. This is called before every visual-servoing IK solve so the Jacobian solver starts from the robot's actual live configuration.

The laptop-side bridge (`joint_target_tcp_bridge_node.py`) retries each command up to three times with a 50 ms delay between attempts and only proceeds once the Pi acknowledges with `"ok": true`.

#### Suction Server (port 5018)

The suction server runs `suction_tcp_server.py` on the Pi. It controls the two relay channels described in Section 2 via the Raspberry Pi GPIO (BCM pin 20 for the pump relay, pin 21 for the valve relay). The relays are active-low, so the server drives each pin LOW to energize the relay and HIGH to de-energize it. Pins are initialized HIGH on startup so both relays start in the off state. The server handles four types of requests:

- **Suction on** ("type": "set", "pump": true). The pump relay is energized (pin 20 LOW) and the valve relay remains de-energized (pin 21 HIGH, valve closed). Negative pressure builds up and the suction cup grips the object. This command is sent just before the visual-servoing descend so the cup is already pulling when it makes contact.
- **Suction off** ("type": "set", "pump": false). The pump relay is de-energized (pin 20 HIGH). The valve remains closed. This is the idle state used when the arm is moving between poses without holding an object.
- **Release** ("type": "release", "pulse\_s": 0.3). The pump is switched off (pin 20 HIGH) and the valve is briefly opened (pin 21 LOW) for the specified pulse duration, then closed again (pin 21 HIGH). Opening the valve vents the tube and breaks the vacuum, causing the object to drop. After the pulse the server restores both relays to the off state.
- **State query** ("type": "get\_state"). Returns the current on/off state of both the pump and valve, the BCM pin numbers, and their current logic levels. Used for diagnostics.

Every response from the suction server echoes the resulting hardware state (pump on/off, valve open/closed, pin levels) so the laptop can confirm the relay transition occurred as expected.

## 5.2 ROS 2 Node Graph

The workspace contains seven ROS 2 nodes split across the two packages. Table 1 summarises their roles, and Figure 4 shows the complete communication graph.

| Node                            | Publishes                       | Subscribes                      | TCP        |
|---------------------------------|---------------------------------|---------------------------------|------------|
| aruco_extrinsic_calibrator      | —                               | —                               | —          |
| top_face_center_node            | /perception/suction_point_world | —                               | —          |
| visual_servoing_perception_node | /visual_servoing/move_cm        | —                               | —          |
| qp_ik_node                      | /myarm/joint_targets            | /perception/suction_point_world | —          |
| joint_target_tcp_bridge_node    | —                               | /myarm/joint_targets            | 5017       |
| visual_servoing_motion_node     | —                               | /visual_servoing/move_cm        | 5017, 5018 |
| task_automation_node            | —                               | —                               | 5017, 5018 |

Table 1: ROS 2 nodes and their communication roles.

**Extrinsic calibration.** `aruco_extrinsic_calibrator` reads camera frames, detects ArUco markers, and writes the solved camera-to-world transform to `~/ros/myarm_camera_extrinsic.json`. It has no publishers and makes no TCP calls. `top_face_center_node` loads this file at startup to project depth points into the world frame; no other node reads it.

**Modular pipeline.** `top_face_center_node` runs the HSV and depth-based top-face detector and publishes the suction point as a `PointStamped` on `/perception/suction_point_world`. `qp_ik_node` subscribes to that topic, solves the pre-grasp IK, and publishes the resulting seven joint angles as a `JointState` on `/myarm/joint_targets`. `joint_target_tcp_bridge_node` subscribes to that topic and forwards each command to the joint server on TCP port 5017.

**Visual-servoing pipeline.** `visual_servoing_perception_node` runs the pixel-space residual error estimator and publishes an XY correction in centimetres as a `Vector3`. `visual_servoing_motion_node` subscribes to that topic, reads the arm’s live joint configuration from TCP port 5017, solves the corrective NR IK, and sends the descend, lift, and drop commands over TCP port 5017 (joint targets) and TCP port 5018 (suction pump and valve).

**Standalone pipeline.** `task_automation_node` implements the full multi-cube grasp loop without using any ROS topics. It drives perception internally, calls TCP port 5017 for all arm moves, and calls TCP port 5018 for suction control, operating as a self-contained script that only needs the two TCP servers to be running on the Pi.

Figure 4 summarises the complete communication architecture.

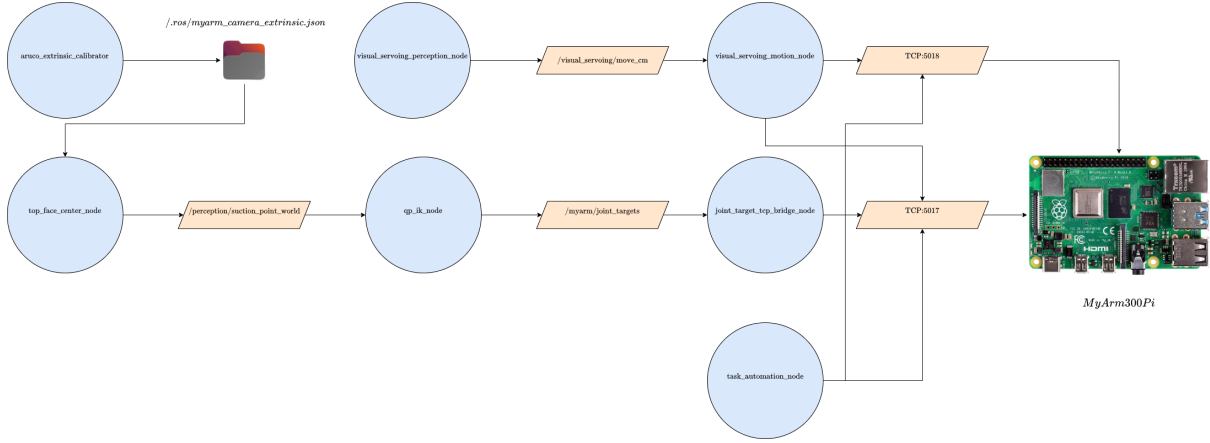


Figure 4: System communication architecture. Circles are ROS 2 nodes, parallelograms are ROS 2 topics or TCP endpoints.

## 6 Results and Discussion

We evaluated the system across a large number of experiments spanning many different scene configurations. These ranged from easy layouts, with the cubes lying flat on the table and well separated, to deliberately harder ones in which cubes were rotated at arbitrary angles, placed close together, or partially occluding one another from the camera’s viewpoint.

In the favorable scenarios—cubes flat on the table and spaced far enough apart that the camera could cleanly distinguish one cube from another—the system performed very reliably, achieving grasp success rates of over 90%. In the more difficult scenarios, where the scene was more cluttered and the cubes were rotated or mutually occluding, the system would occasionally miss a target, typically because the perception stage could not cleanly resolve the top face of a partially hidden or awkwardly oriented cube. Importantly, however, even these failed attempts were rarely fatal to the overall task: a grasp attempt that brushed or nudged a cube would frequently rearrange the scene, separating clustered cubes or knocking a rotated cube into a flatter orientation, so that on a subsequent round the robot could detect and grasp it successfully. In this way the automated loop tended to make an initially infeasible scene gradually feasible, and was usually able to clear all the cubes given enough attempts.

It is worth emphasizing just how demanding the precision requirement is for this particular task. The diameter of the suction cup is approximately equal to the edge length of the cube, and reliable suction required almost the entire area of the cup to be in contact with the cube’s top face. There is therefore very little tolerance for error: a positioning error of only a few millimeters means the cup overhangs the edge of the cube, breaks its seal, and fails to lift it. This is precisely why the multi-stage approach—accurate extrinsic calibration, table-anchored top-face estimation, and a final visual-servoing correction with the measured hardcoded offsets—was necessary, rather than relying on the pre-grasp pose alone. Every stage of the pipeline exists to drive the suction cup onto that small target area with the required millimeter-level accuracy.

A live demonstration of the robot performing a full run of the task automation pipeline (Section 4.3) can be found [here](#).